Titulación: GRADO INGENIERÍA INFORMÁTICA Año Académico: 2024/25 -- Curso: 3º Asignatura: Sistemas Distribuidos	uc3m | Universidad Carlos III de Madrid

Alumno/a:	**Arturo Hernanz Carrasco-Muñoz**	**NIA:**	**100495850**



Memoria Ejercicio 3: RPC

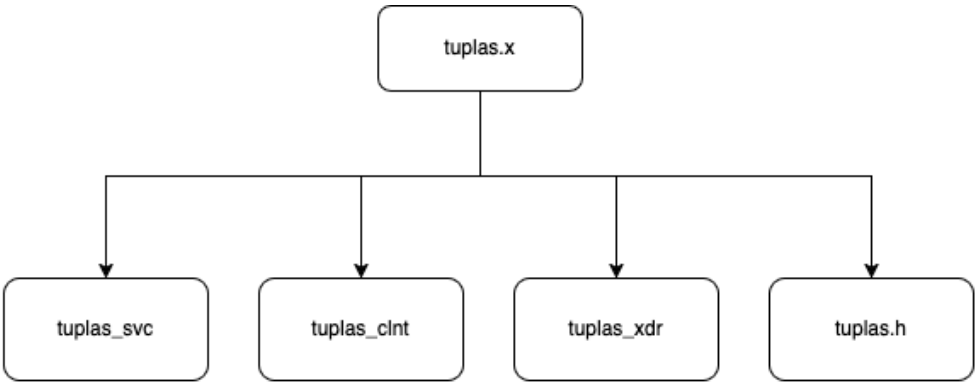
Titulación: GRADO INGENIERÍA INFORMÁTICA Año Académico: 2024/25 -- Curso: 3º Asignatura: Sistemas Distribuidos	uc3m | Universidad Carlos III de Madrid

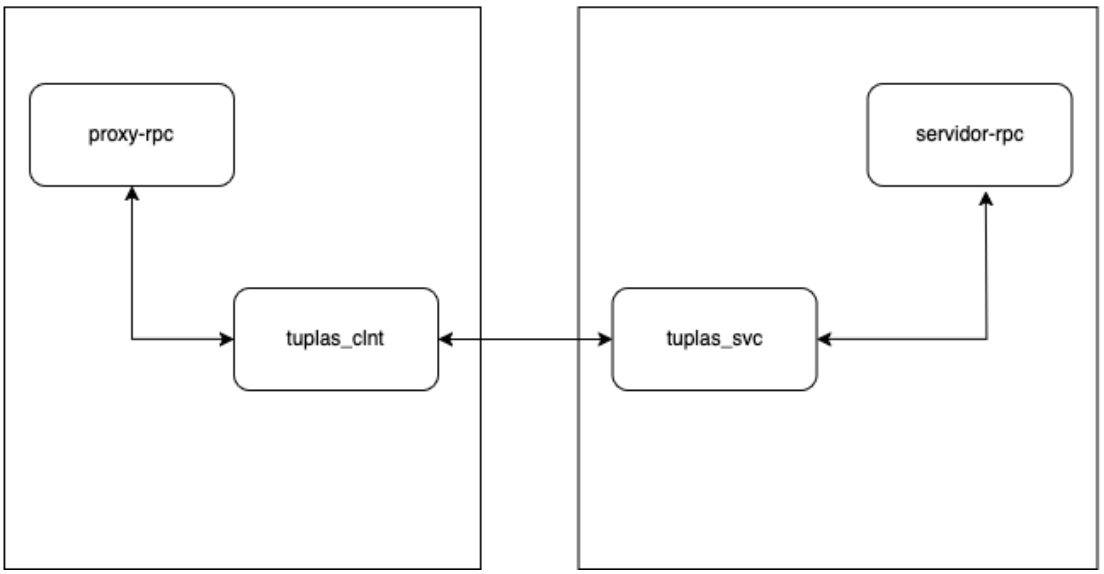
Índice:

1. Decisiones de diseño destacadas
2. Compilación
3. Plan de pruebas
4. Estadísticas

1. Decisiones de diseño destacadas:

Para el desarrollo de esta práctica se ha hecho uso del comando "***rpcgen -CNM tuplas.x***" que genera los ficheros necesarios para la arquitectura RPC con las flags de generar el código en formato "***ANSI C***" más moderno (***-C***), varios argumentos por función (***-N***) y permite el funcionamiento multihilo, necesario para este ejercicio ya que se prevé la presencia de varios clientes simultáneos (***-M***).
Este comando genera, del fichero "***tuplas.x***" que define las funciones, los stubs del cliente y el servidor, así como, el "traductor" XDR y el fichero con las cabeceras.





El código sigue el siguiente comportamiento:

El cliente llama a las funciones de ***proxy-rpc.c*** a través de la ***API***, donde mediante RPC, se le envían los datos a ***servidor-rpc.c***.
El proxy hace uso de las llamadas disponibles en el stub del cliente (***tuplas_clnt***) que, a través de sockets, se comunica con el stub del servidor (***tuplas_svc***) que llama a las funciones disponibles en el servidor.

Este llama a las funciones de ***claves.c*** que devuelven 0 o -1 dependiendo de si han funcionado bien o si ha ocurrido algún error.
A continuación, el servidor envía al proxy (realizando el proceso inverso) la respuesta de estas funciones.

A continuación se detallan un poco más el comportamiento de cada componente y se explican algunas decisiones tomadas:

- **Estructura de datos seleccionada:**
  Para el desarrollo de este ejercicio se ha decidido hacer uso de listas enlazadas como estructura de datos debido a la facilidad de implementación así como la posibilidad de hacer uso de la cantidad de memoria que se necesite en cada caso.
  Aunque tienen como desventaja la complejidad O(n) para búsqueda y, en el caso de este ejercicio, para inserción, ya que, hay que asegurar que no se insertan dos tuplas para la misma clave.

- **claves.c:**
  Para el desarrollo del fichero de funciones se han implementado las funciones teniendo en cuenta que la estructura de datos es una lista enlazada.

- **proxy-rpc.c:**
  Para el desarrollo del proxy del cliente se ha creado una función por cada función de ***claves.c*** que completa los campos de una estructura predefinida en el fichero ***estructuras.h*** que se le pasa a la función que se comunica con el servidor.

- **servidor-rpc.c:**
  Para el desarrollo del servidor se han desarrollado las funciones que el cliente llamaría de forma remota, adaptándose al esquema y comportamiento descritos previamente.
  Resalto que la función ***get_value*** se comprueba si ha fallado o no, en caso de error, se modifica el valor del ***key*** cambiandolo por ***-1*** y se devuelven los valores nulos salvo el valor mencionado previamente.
  En el resto de funciones se devuelve 0 o -1.

2. Compilación:

Para la compilación y generación de los ejecutables del cliente y el servidor se ha hecho un makefile en el que se compilan los diferentes ficheros y se crean los ejecutables.

Se crea un ejecutable a través del fichero ***servidor-rpc*** y uno por cada fichero ***app-cliente*** usados para las pruebas, para estos últimos es necesario hacer uso de la librería que llama a las funciones del fichero ***proxy-rpc*** y se encargan de enviar y recibir la información del servidor.

Para la generación de los ficheros RPC se hace uso del comando "***rpcgen -CNM tuplas.x***" explicado previamente y se modifican los ficheros correspondientes para asegurar un comportamiento adecuado del sistema.

3. Plan de pruebas:

Para probar el buen funcionamiento de este proyecto se han diseñado 7 tipos de clientes, uno por cada función a probar y uno extra para la función ***set_value***, ya que, para las pruebas es útil tener al menos dos tuplas diferentes en la base de datos.

Para probar todos estos clientes se han diseñado tres tipos de pruebas:

1. Pruebas que deben funcionar:
   - **1.1.** **set_modify_get**: Se inserta un valor en la base de datos, luego se modifica y por último se obtienen los valores de la tupla.
   - **1.2.** **set_get:** Se inserta un valor en la base de datos y luego se obtienen los valores de la tupla.
   - **1.3.** **set_get–exist:** Se inserta un valor en la base de datos y luego, de forma concurrente, se obtienen los valores de la tupla y se comprueba su existencia.
   - **1.4.** **set_get–get–get:** Se inserta un valor en la base de datos y se obtienen tres veces al mismo tiempo los valores de la tupla, verificando el buen funcionamiento de la concurrencia.
   - **1.5.** **set–set:** Se insertan dos valores diferentes en la base de datos al mismo tiempo.

2. Pruebas que deben fallar:
   - **2.1.** **destroy_get:** Se destruye la base de datos y después se intenta obtener los datos de una tupla.
   - **2.2.** **set_delete:** Se inserta un valor y después se intenta eliminar un valor que no existe en la base de datos.
   - **2.3.** **set_exist:** Se inserta un valor y después se intenta verificar la existencia de un valor que no existe en la base de datos.
   - **2.4.** **set_get:** Se inserta un valor y después se intenta obtener los valores de una tupla asociada a un valor que no existe en la base de datos.

2.5. **set_modify:** Se inserta un valor y después se intenta modificar un dato de la tupla asociada a un valor que no existe en la base de datos.

3. Pruebas que debido al uso de ***threads*** pueden o no fallar:
   - **3.1.** **set–delete:** Se inserta un valor a la base de datos y al mismo tiempo se borra, no se puede asegurar que la inserción se hará antes que el borrado.
   - **3.2.** **set–destroy:** Se inserta un valor a la base de datos y al mismo tiempo se destruye la base de datos, no se puede asegurar que la inserción se hará antes que la destrucción de la base.
   - **3.3.** **set–exist:** Se inserta un valor a la base de datos y al mismo tiempo se comprueba su existencia, no se puede asegurar que la inserción se hará antes que la comprobación.
   - **3.4.** **set–get:** Se inserta un valor a la base de datos y al mismo tiempo se obtienen los valores de la tupla, no se puede asegurar que la inserción se hará antes que la obtención.
   - **3.5.** **set–modify:** Se inserta un valor a la base de datos y al mismo tiempo se modifica, no se puede asegurar que la inserción se hará antes que la modificación.

Para ejecutar estas pruebas hay que ejecutar el fichero "***test_all.sh***", que ejecuta los tres conjuntos de pruebas, siguiendo los siguientes pasos:

1. Ejecutar el makefile la primera vez que se realizan las pruebas:

```
make
// Exportar la libreria (Solo se hace la primera vez)
export LD_LIBRARY_PATH=.
```

2. En una terminal se ejecuta el lado del servidor:

```
./servidor
```

3. En otra terminal se ejecuta el lado del cliente:

```
// Ejecutar todas las pruebas
// En caso de haber algún error de permisos ejecutar:
// chmod +x <carpeta_de_pruebas>/*
./test_all.sh
```

En mi caso, al probar el ejercicio en ***guernika*** abrí dos terminales e inicié sesión en ambas, lo que daba error de lectura de colas, ya que, cada terminal tenía un espacio diferente.
En este caso hay que seguir estos pasos:

1. Ejecutar el makefile la primera vez que se realizan las pruebas:

```
make
// Exportar la libreria (Solo se hace la primera vez)
export LD_LIBRARY_PATH=.
```

2. Ejecutar el servidor para que esté en segundo plano:

```
./servidor &
```

3. Ejecutar el lado del cliente:

```
// Ejecutar todas las pruebas
// En caso de haber algún error de permisos ejecutar:
// chmod +x <carpeta_de_pruebas>/*
./test_all.sh
```

4. Estadísticas:

Para evaluar el comportamiento de la práctica a nivel general se ha hecho uso de las herramientas "***Perf Stat***" y "***Valgrind***", arrojando los siguientes resultados:

- "***Perf Stat***": Se ha evaluado tanto para el servidor como para el fichero de test.
  - ***Servidor***:

```
Performance counter stats for './servidor':

            8,93 msec task-clock                       #    0,000 CPUs utilized
             111      context-switches                 #   12,432 K/sec
              20      cpu-migrations                   #    2,240 K/sec
             223      page-faults                      #   24,976 K/sec
  <not supported>      cycles
  <not supported>      instructions
  <not supported>      branches
  <not supported>      branch-misses

    118,133265816 seconds time elapsed

      0,000000000 seconds user
      0,009782000 seconds sys
```

  - ***Fichero de test***:

```
Performance counter stats for './test_all.sh':

          197,94 msec task-clock                       #    0,003 CPUs utilized
           1.477      context-switches                 #    7,462 K/sec
             156      cpu-migrations                   #  788,115 /sec
          19.036      page-faults                      #   96,170 K/sec
  <not supported>      cycles
  <not supported>      instructions
  <not supported>      branches
  <not supported>      branch-misses

     75,575565857 seconds time elapsed

      0,122284000 seconds user
      0,082457000 seconds sys
```

- "***Valgrind***": Solo se ha evaluado para el fichero de test, ya que, al detener la ejecución del servidor mediante "***ctrl + C***" los resultados se alteraban.
  - ***Fichero de test***: Es destacable que no hay goteos de memoria

```
==101549==
==101549== HEAP SUMMARY:
==101549==     in use at exit: 61,145 bytes in 711 blocks
==101549==   total heap usage: 5,873 allocs, 5,162 frees, 3,360,865 bytes allocated
==101549==
==101549== LEAK SUMMARY:
==101549==    definitely lost: 0 bytes in 0 blocks
==101549==    indirectly lost: 0 bytes in 0 blocks
==101549==      possibly lost: 0 bytes in 0 blocks
==101549==    still reachable: 61,145 bytes in 711 blocks
==101549==         suppressed: 0 bytes in 0 blocks
==101549== Reachable blocks (those to which a pointer was found) are not shown.
==101549== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==101549==
==101549== For lists of detected and suppressed errors, rerun with: -s
==101549== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```